

AI-STUDY-LAB · 第 0 周装机 → P0 / P1 / P2

Mac mini 上手手册

从打开终端，到在租来的 GPU 上跑完一次 ACT 训练。
每一步都写清楚：敲什么、应该看到什么、没看到怎么办。

目标机器 Mac mini M4 / 16GB 统一内存

配套 PLAN.md 第 0 周 (2026-09-24 ~ 09-27) · Phase 0 起于 2026-09-28

版本 第一版 · 2026-09-24

目录

00 这份文档怎么用

三段式、时间盒纪律、60 秒规则

01 先认识你的机器

开终端 · 五个动作 · 自检三条命令 · 能做什么不能做什么

02 装地基

Xcode CLT · Homebrew · uv · Python 3.12

03 仓库一：micrograd

考环境隔离 · 时间盒 1h

04 仓库二：MuJoCo

考系统依赖与渲染 · 时间盒 2h

05 仓库三：LeRobot

考研究仓库全套 · 时间盒 4h

06 P1：MuJoCo 机械臂 IK

2 周 · 全程本地免费

07 P2：租 GPU 跑真训练

3 周 · 本地做什么、云上做什么

08 卡住了怎么办

60 秒规则的操作版 · 高频报错对照表

09 命令速查卡

一页，可单独打印

00 · 这份文档怎么用

这份手册解决的是一个很具体的问题：你知道今天该跑 micrograd，但不知道第一条命令敲什么。

学习计划（`PLAN.md`）在 9 月初就写好了，课程表、每日推送、债务规则全都齐了，然后三周过去，一条笔记都没产出。原因不是计划不好，是计划和你的键盘之间缺了一层。这份文档就是那一层。

每一步都是三段式

全书所有操作都长这个样子：

① 敲什么

```
$ uname -m
```

② 应该看到
`arm64`。这说明你在 Apple Silicon 上，全书的命令都按这个写。

③ 没看到怎么办
如果输出是 `x86_64`，你要么在 Intel 机器上，要么终端跑在 Rosetta 转译下。第 1 章末尾有对照处理。

第二段最重要。不知道「对」长什么样，你就永远分不清「还在装」和「已经卡住了」——这是新手浪费整晚的头号原因。

两条纪律

纪律	怎么做
装机不是学习	第 1-2 章属于「第 0 周」（09-24 ~ 09-27），在正式开课前做完。不要让配环境吃掉 W1 的学习日——那会让你第一周就产生「我在学但什么都没学到」的感觉。
60 秒规则	任何报错，先自己读 60 秒再问 AI。这 60 秒练的是别人替你练不了的肌肉。第 8 章是这条规则的操作版。踩坑日志里专门有一列记这个时长，就是为了让这条规则可被自己审计。

时间盒到了就停
每个仓库都标了时间盒（1h / 2h / 4h）。到点没跑通，停下来把状态记进踩坑日志，第二天接着来。不要硬扛到半夜，也不要为了「跑通」去抄一个别人的现成环境——那等于把这次训练作废。卡住的过程本身就是 P0 的交付物。

标记的含义

命令后面会有两种标记：

已验证 这条信息在写文档时对着官方来源或包索引核对过（比如 PyPI 上某个包的版本和 wheel 平台）。

未实测 没在真机上跑过 —— 写这份文档的机器是 Linux 云容器，不是你的 Mac，而且出网受限。这类命令出自官方文档但可能已经变了。凡是和文档写的不一致，一律以你终端里的实际输出为准，并把差异记下来，第二版会修。

第一版必然有要修的地方

一份没人跑过的手册不可能全对。你照着走一遍、把对不上的地方记下来，这份手册的第二版才真正有价值。这个过程本身也是 PO 在练的东西：照着别人的文档跑，然后发现文档哪里错了。

01 · 先认识你的机器

1.1 打开终端

按 `Command + 空格` 打开 Spotlight，输入 `Terminal`（或中文系统里的「终端」），回车。

你会看到一个窗口，最后一行大概长这样：

```
jiangnan@Mac-mini ~ %
```

这一行叫提示符。`%` 后面就是你敲命令的地方（zsh 用 `%`，bash 用 `$`；macOS 现在默认是 zsh）。本书里所有命令前面的 `$` 都是提示符的意思，不要把它敲进去。

1.2 五个动作，够用了

动作	干什么
<code>pwd</code>	我现在在哪个文件夹。迷路了就敲它。
<code>ls</code>	这个文件夹里有什么。 <code>ls -la</code> 连隐藏文件一起列。
<code>cd 路径</code>	进到某个文件夹。 <code>cd ..</code> 是回上一层， <code>cd ~</code> 是回家目录。
Tab 键	最重要的一个。打了一半按 Tab 自动补全。补不出来说明路径打错了——这能省掉一半的「文件找不到」。
<code>Control + C</code>	强行中止当前命令。卡住了、跑飞了，按它。

一个能省很多事的习惯

路径里有空格或中文时，用引号包起来：`cd "~/Documents/WorldEngine Claude folder"`。不包的话 shell 会把空格当成两个参数的分隔符，报一个看起来毫不相干的错。

1.3 自检：你的机器是什么

① 敲什么

```
# 芯片架构
$ uname -m

# macOS 版本
$ sw_vers

# 芯片型号与内存
$ system_profiler SPHardwareDataType | grep -E "Chip|Memory|Model Name"
```

② 应该看到

依次是 `arm64` ； `ProductVersion: 15.x` 之类；以及 `Chip: Apple M4` 和 `Memory: 16 GB` 。

③ 没看到怎么办

`uname -m` 输出 `x86_64` 但你确定是 M 系列芯片 → 终端跑在 Rosetta 下。在「访达 → 应用程序 → 实用工具」里右键 Terminal → 显示简介 → 取消勾选「使用 Rosetta 打开」，重开终端。

`system_profiler` 那条跑得慢是正常的（要几秒）。

1.4 M4 / 16GB 能做什么，不能做什么

这张表决定了后面哪一章你可以慢慢来、哪一章必须租卡。提前知道边界，比撞上去再查省一整晚。

任务	本机	说明
micrograd（第 3 章）	✅ 轻松	纯 Python，没有二进制依赖，CPU 秒出。
MuJoCo 仿真与 IK（第 4、6 章）	✅ 轻松	官方有 arm64 wheel 已验证 。机械臂级别的仿真 M4 绰绰有余。
LeRobot 装环境、看数据集、读代码（第 5 章）	✅ 可以	依赖树大，装的时候要等，但能装上。
改超参、跑几十步看曲线动起来	⚠️ 勉强	能跑，慢。目的是看懂流程，不是出指标。
ACT / Diffusion Policy 完整训练（第 7 章）	❌ 不要	16GB 统一内存要同时喂 CPU 和 GPU；没有 CUDA，MPS 在这类模型上支持不完整。租卡。第 7 章教你自己算要多久。

为什么不直接给你一个「要跑 X 小时」的数字

因为我不在你的机器上测过，编一个数会害你按错误的预期安排时间。第 7 章给的是测量方法：本地跑 100 步计时，按比例外推。这既得到真实数字，也顺手练了估算——那是研究者的日常功夫。

02 · 装地基

一次性工作，约 40 分钟，其中大半时间在等下载。装完之后第 3–7 章都不要再碰这一章。

要装四样东西，顺序不能换：

装什么	为什么要它
Xcode 命令行工具	提供编译器和 <code>git</code> 。很多 Python 包装的时候要现编译一小部分，没有它就会报一堆看不懂的错。
Homebrew	macOS 的包管理器。装系统级软件用它。
uv	Python 环境和包管理。比 conda 快得多、简单得多，现在是研究仓库的事实标准。
Python 3.12 (Homebrew 版)	全书统一用 3.12。另外第 4 章的 MuJoCo 交互式窗口需要一个 Homebrew 装的 Python，理由在那一章。

2.1 Xcode 命令行工具

① 敲什么

```
$ xcode-select --install
```

② 应该看到

弹出一个图形安装窗口，点「安装」，等 5–15 分钟（取决于网速）。

如果已经装过，会直接提示 `command line tools are already installed` —— 那就是好事，跳到 2.2。

③ 没看到怎么办

装完验证：`git --version` 应该输出一个版本号。如果提示找不到 `git`，重开一个终端窗口再试（PATH 需要新会话才生效）。

2.2 Homebrew

① 敲什么

```
# 官方安装脚本（以 brew.sh 首页为准）
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

未实测

② 应该看到

脚本会先列出它打算做什么，然后要你按回车确认，可能还要输入 Mac 登录密码（输密码时屏幕不会显示任何字符，这是正常的，敲完直接回车）。

结束时打印一段 `Next steps`，里面有两行 `echo ... >> /Users/你的用户名/.zprofile` 和 `eval ...`。

③ 没看到怎么办

装完直接敲 `brew --version` 大概率提示 `command not found` —— 这是 Apple Silicon 上最常见的一个坎，不是装失败了。原因是 Homebrew 装在 `/opt/homebrew`，不在默认 PATH 里。执行上一段里那两行（或者照抄下面这两条），然后重开终端。

① 敲什么（接上，配置 PATH）

```
$ echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
$ eval "$(/opt/homebrew/bin/brew shellenv)"
$ brew --version
```

② 应该看到

`Homebrew 4.x.x` 之类的版本号。到这里 Homebrew 就好了。

2.3 uv 与 Python 3.12

① 敲什么

```
$ brew install uv python@3.12
$ uv --version
$ /opt/homebrew/bin/python3.12 --version
```

已验证 (`brew install uv` 是 Astral 官方给出的 macOS 安装方式之一)

② 应该看到

两个版本号，第二个是 `Python 3.12.x`。
装的过程会拉不少依赖，几分钟很正常。

③ 没看到怎么办

`python3.12` 找不到 → 用 `brew --prefix python@3.12` 看它实际装在哪，路径可能和这里写的不同。
下载慢或超时 → 大概率是网络问题，换个网络或稍后重试；Homebrew 的下载不走代理时经常很慢。

为什么全书钉死 PYTHON 3.12

LeRobot 0.6 起要求 Python ≥ 3.12 已验证 (PyPI 上 `requires_python` 写着 `>=3.12`)，而 MuJoCo 3.10 到 3.15 都支持。全程用同一个版本，可以少掉一整类「装到一半说版本不合」的坑。
如果你看过本仓库的 `ENGINEERING.md` 早期版本写的是 3.11 —— 那是写在确认 LeRobot 要求之前，已经改了。

2.4 一个必须理解的概念：虚拟环境

后面每一章开头都会建一个「虚拟环境」。花两分钟搞懂它，能省掉后面 80% 的怪问题。

问题是这样的：micrograd 可能要 A 库的 1.0 版，LeRobot 要 A 库的 2.0 版。装在一起必然打架。虚拟环境就是给每个项目一个独立的小房间，各装各的，互不影响。

命令	意思
<code>uv venv --python 3.12</code>	在当前文件夹建一个叫 <code>.venv</code> 的小房间
<code>source .venv/bin/activate</code>	走进这个房间。之后提示符前面会多一个 <code>(.venv)</code>
<code>uv pip install 包名</code>	把包装进当前房间
<code>deactivate</code>	走出房间

最高频的一个错

开了新终端窗口就是新会话，之前的 `activate` 不算数了。症状是「我明明装过了却说找不到模块」。
判断方法：看提示符前面有没有 `(.venv)`。没有就重新 `source .venv/bin/activate`。这一条会在第 8 章再出现一次，因为它真的很常见。

2.5 把学习仓库拉下来

① 敲什么

```
$ mkdir -p ~/code && cd ~/code
$ git clone https://github.com/SoaringWillow/ai-study-lab.git
$ cd ai-study-lab
$ ls
```

② 应该看到

一串文件名：`PLAN.md ENGINEERING.md README.md notes papers predictions experiments schedule bot docs`。

这就是你的学习仓库，后面所有笔记、论文笔记、踩坑日志都写在这里。

第 0 周到这里就结束了

地基装完，下面三章分别是 P0 的三个仓库。它们排在 W1 的动手日和之后 —— 今天不用往下做。

真要现在试试手，跳到第 3 章的 `micrograd`，它只要 1 小时，而且没有任何二进制依赖，是最安全的第一次。

03 · 仓库一：micrograd

时间盒 1 小时。这一关考的是「环境隔离」。

为什么第一个是它：纯 Python，没有二进制依赖，不用下数据集，不碰 GPU。如果连它都卡住，问题一定在环境本身，不在别的地方——这种「一次只可能错一件事」的起点，是排查能力的地基。

3.1 拿下来并建房间

① 敲什么

```
$ cd ~/code
$ git clone https://github.com/karpathy/micrograd.git
$ cd micrograd
$ uv venv --python 3.12
$ source .venv/bin/activate
$ uv pip install -e .
```

② 应该看到

`uv venv` 后提示创建了 `.venv`；`activate` 之后提示符前面出现 `(micrograd)` 或 `(.venv)`；最后一条打印 `Installed 1 package` 之类。

③ 没看到怎么办

`uv pip install -e .` 报 `No such file or directory` 或找不到项目文件 → 你多半没在 `micrograd` 目录里。敲 `pwd` 确认，敲 `ls` 应该能看到 `setup.py`。
提示符前面没有 `(...)` → `activate` 没生效，重新 `source .venv/bin/activate`。

3.2 出第一个数字

不要一上来开 notebook。先用一个纯脚本确认能 import、能反向传播。

① 敲什么

```
# 用 cat 写一个小脚本（整段粘贴，最后那行单独的 EOF 也要）
$ cat > smoke.py <<'EOF'
from micrograd.engine import Value

a = Value(-4.0)
b = Value(2.0)
c = a + b
d = a * b + b**3
c += c + 1
c += 1 + c + (-a)
d += d * 2 + (b + a).relu()
d += 3 * d + (b - a).relu()
e = c - d
f = e**2
g = f / 2.0
g += 10.0 / f

print("forward g =", round(g.data, 4))
g.backward()
print("dg/da =", round(a.grad, 4))
print("dg/db =", round(b.grad, 4))
EOF

$ python smoke.py
```

未实测（脚本取自 micrograd README 的示例）

② 应该看到

三个浮点数。README 给的参考值是 `24.7041` / `138.8338` / `645.5773`。

数字对不对是次要的，关键是三行都打印出来了——这说明包装对了、`backward()` 真的沿着计算图跑了一遍。这就是你的「第一个数字」。

③ 没看到怎么办

`ModuleNotFoundError: No module named 'micrograd'` → 三种可能，按这个顺序查：

1. 提示符前面有没有 `(...)`？没有就是环境没激活。
2. `uv pip install -e .` 真的跑成功了吗？重跑一次看输出。
3. 你是不是在别的目录？`pwd` 确认。

注意这个错的模块名正好等于仓库名——这是「忘了 editable install」的典型指纹，不要先去怀疑 Python 版本。（W1 D1 那道题问的就是这个。）

3.3 再看 loss 真的下降

仓库里的 `demo.ipynb` 训练一个两层 MLP 做二分类，能看到 loss 一路降下来。它要几个额外的包。

① 敲什么

```
$ uv pip install numpy matplotlib scikit-learn jupyter
$ jupyter notebook demo.ipynb
```

未实测

② 应该看到

浏览器自动打开一个页面。从上往下逐个单元格按 `Shift + Enter` 执行。

训练那个单元格会打印一串 `step 0 loss 0.8xxx, accuracy 50%` 这样的行，loss 应该一路往下走、accuracy 往上走。最后画出一张决策边界图。

③ 没看到怎么办

跑不起来 jupyter 不影响这一关通过 —— 3.2 的三个数字已经达标了。把问题记进踩坑日志，往下走。终端要收回控制权：回到终端按 `Control + C` 两次。

这一步真正的功课：逐行可复述

打开 `micrograd/engine.py`，找到 `backward()` 那个方法（约 20 行）。它做两件事：先拓扑排序建出计算图的倒序，再沿着倒序逐个调用每个节点的 `_backward`。

合上屏幕，用自己的话复述这两步。说不出来就再读一遍。`CLAUDE.md` 的规矩是「说不出某一行在干什么，这段代码不许 commit」—— 这里是你第一次执行这条规矩。这 100 行是整个深度学习的最小可理解内核，值得读三遍。

收尾

把今天每一个坑记进 `~/code/ai-study-lab/experiments/P0-toolchain/踩坑日志.md`：报错最后一行原文 / 你自己读了多久 / 真实原因 / 解法 / 下次怎么更快认出它。
跑通了但日志是空的，这一天不算完成。

04 · 仓库二：MuJoCo

时间盒 2 小时。这一关考的是「系统依赖与渲染后端」——和 Python 包管理完全无关的另一类坑。

这一章分两段：先跑无图形界面的（必通），再试交互式窗口（可选）。这个顺序是刻意的，理由在 4.3。

4.1 装上

① 敲什么

```
$ cd ~/code && mkdir -p mujoco-play && cd mujoco-play
$ uv venv --python /opt/homebrew/bin/python3.12
$ source .venv/bin/activate
$ uv pip install mujoco
$ python -c "import mujoco; print(mujoco.__version__)"
```

已验证 (PyPI 上 mujoco 3.14.0 有 `macosx_11_0_arm64` 轮子，覆盖 Python 3.10–3.15)

② 应该看到

一个版本号，比如 `3.14.0`。装的时候不应该出现编译过程——有现成的 arm64 轮子，直接下载解压。看到大段 `Building wheel...` 说明轮子没匹配上，检查 Python 版本。

注意这里的 `VENV` 和第 3 章不一样

第 3 章用的是 `uv venv --python 3.12` (uv 自己下的 Python)，这里指定了 Homebrew 的 `/opt/homebrew/bin/python3.12`。原因在 4.3 的 `mjpython`——它和 uv 自带的 Python 不兼容。先这么做，理由马上说。

4.2 无界面：出第一个数字和第一帧图

不需要任何模型文件——直接在代码里写一个最小场景（一个盒子从空中掉到地面）。

① 敲什么

```
$ cat > drop.py <<'EOF'
import mujoco

XML = """
<mujoco>
  <worldbody>
    <light pos="0 0 3"/>
    <geom type="plane" size="2 2 .1"/>
    <body pos="0 0 1">
      <freejoint/>
      <geom type="box" size=".1 .1 .1" rgba="1 .4 .2 1"/>
    </body>
  </worldbody>
</mujoco>
"""

model = mujoco.MjModel.from_xml_string(XML)
data = mujoco.MjData(model)

print("每步仿真多少秒 timestep =", model.opt.timestep)
print("起始高度 z =", round(float(data.qpos[2]), 4))

for _ in range(200):
    mujoco.mj_step(model, data)

print("200 步之后 z =", round(float(data.qpos[2]), 4))
print("仿真时间 =", round(data.time, 4), "秒")
EOF

$ python drop.py
```

② 应该看到

四行输出。timestep 默认是 0.002 (2 毫秒)；起始 z 是 1.0 左右；200 步之后 z 明显变小 (盒子掉下来了，最后停在盒子半高 0.1 附近)；仿真时间 = $200 \times 0.002 = 0.4$ 秒。这就是这一关的「第一个数字」。盒子会掉，说明物理引擎真的在算。

③ 没看到怎么办

z 完全没变 → 检查 <freejoint/> 有没有漏掉；没有关节的物体是焊死在世界上的，不会动。
XML 报解析错 → 多半是粘贴时尖括号被转义了，对照检查。

顺手理解三个概念（P1 要用）

timestep：每次 `mj_step` 推进多少仿真时间。调小更准但更慢。

接触：盒子落地后停住，是接触求解器在算法向力。把 `size` 改小、把初始高度改高，观察落地瞬间的行为变化。

求解器：`model.opt` 里还有迭代次数等参数，决定接触算得多精细。

改一个参数、跑、看数字怎么变 —— 这比读十页文档管用。

4.3 交互式窗口：一个必须知道的坑

MuJoCo 有个能用鼠标拖动视角的实时窗口。在 macOS 上它不能用普通的 `python` 启动，必须用随包安装的 `mjpython`。

原因：macOS 要求图形渲染必须在主线程上跑，`mjpython` 就是为满足这个约束而存在的启动器。 已验证
(MuJoCo 官方文档)

① 敲什么

```
$ cat > watch.py <<'EOF'
import time, mujoco, mujoco.viewer

XML = open("scene.xml").read() if False else """
<mujoco>
  <worldbody>
    <light pos="0 0 3"/>
    <geom type="plane" size="2 2 .1"/>
    <body pos="0 0 1">
      <freejoint/>
      <geom type="box" size=".1 .1 .1" rgba="1 .4 .2 1"/>
    </body>
  </worldbody>
</mujoco>
"""

model = mujoco.MjModel.from_xml_string(XML)
data = mujoco.MjData(model)

with mujoco.viewer.launch_passive(model, data) as viewer:
    while viewer.is_running():
        mujoco.mj_step(model, data)
        viewer.sync()
        time.sleep(0.002)

EOF

# 注意：是 mjpgpython, 不是 python
$ mjpgpython watch.py
```

② 应该看到

弹出一个窗口，盒子掉在地上。鼠标左键拖动是旋转视角，滚轮缩放。关窗口即退出。

③ 没看到怎么办

用 `python watch.py` 跑 → 必然失败，报错会提到必须用 `mjpgpython`。这是设计如此，不是你装错了。
`mjpgpython` 报找不到 `libpython3.12.dylib` → 这是 `mjpgpython` 和 `uv` 自带 Python 的已知冲突
(MuJoCo issue #1923, 至今未关) [已验证](#)。这正是 4.1 让你用 `--python`
`/opt/homebrew/bin/python3.12` 建环境的原因。如果你当时用的是 `uv venv --python 3.12`，删掉
`.venv` 重建：

```
$ deactivate; rm -rf .venv
$ uv venv --python /opt/homebrew/bin/python3.12
$ source .venv/bin/activate && uv pip install mujoco
```

还是打不开也没关系 —— 这一关照样算过。4.2 的数字已经达标，交互窗口是锦上添花。把现象记进踩坑日志，往下走。不要在这里烧掉一晚上。

4.4 为第 6 章准备：拉机械臂模型库

① 敲什么

```
$ cd ~/code  
$ git clone https://github.com/google-deepmind/mujoco_menagerie.git
```

② 应该看到

clone 会有点大（几百 MB，含网格模型），等几分钟。完成后 `ls mujoco_menagerie` 能看到一堆机器人目录：`franka_emika_panda`、`universal_robots_ur5e`、`trs_so_arm100` 等等。第 6 章会用其中一个。

05 · 仓库三：LeRobot

时间盒 4 小时。这一关考的是「研究仓库全套」：大依赖树、版本互锁、数据集下载。前两个仓库是热身，这个才是本体——也是最接近你以后每次读新论文官方代码时会遇到的那种复杂度。

5.1 两条必须先知道的约束

约束	后果
Python 必须 ≥ 3.12 已验证	LeRobot 0.6 的 <code>requires_python</code> 写着 <code>>=3.12</code> 。用 3.11 建环境会直接装不上，而且报错信息不一定说人话。
不要自己先装 torch 已验证	LeRobot 锁 <code>torch<2.12</code> ，而 <code>pip install torch</code> 默认给你最新的 2.14。先装 torch 再装 lerobot，解析器要么报冲突、要么把你刚装的 torch 卸了重装——白等半小时。让 lerobot 自己带 torch 下来。

5.2 装上

① 敲什么

```
$ cd ~/code && mkdir -p lerobot-play && cd lerobot-play
$ uv venv --python 3.12
$ source .venv/bin/activate
$ uv pip install "lerobot[dataset]"
```

② 应该看到

屏幕滚很久。这一步下几个 GB、花十几分钟是正常的——torch 本身就有几百 MB。最后打印 `Installed N packages`，N 是三位数。

③ 没看到怎么办

报 `requires-python` 不满足 → 你的 venv 不是 3.12。 `python --version` 确认，不对就 `deactivate; rm -rf .venv` 重建。

卡在某个包下载不动 → 多半是网络。 `Control + C` 中断后重跑，uv 会复用已下好的部分，不会从头来。

报依赖冲突且提到 torch → 你这个环境里之前装过 torch。最省事的办法是删掉 `.venv` 从头来，不要试图手工调版本。

5.3 出第一个数字

① 敲什么

```
$ lerobot-info
```

已验证 (LeRobot README 给出的入门命令)

② 应该看到

一份环境报告：lerobot 版本、torch 版本、平台、以及有没有可用的加速设备。在 M4 上应该能看到 MPS 可用（Apple 的 GPU 接口），CUDA 不可用（正常，Mac 上没有 N 卡）。这就是这一关的「第一个数字」。它证明包装好了、能 import、能探测硬件。

③ 没看到怎么办

`command not found: lerobot-info` → 环境没激活（看提示符），或者装的时候其实失败了（往上翻输出找红色的 error）。

5.4 摸一下真实数据

接下来的命令名和参数可能随版本变化，所以这里先教你一个比任何具体命令都有用的习惯：

① 敲什么

```
$ lerobot-train --help
$ lerobot-eval --help
```

② 应该看到

长长的参数列表。不要试图读完，找这几个就够：`--policy.type`（用哪个策略，比如 `act`、`diffusion`）、`--dataset.repo_id`（用哪个数据集）、`--steps` 或类似的训练步数、`--output_dir`。

③ 没看到怎么办

参数名和这里写的不一样 → 以 `--help` 的输出为准，那是你这个版本的真相。顺手把差异记进踩坑日志，这正是「照着别人的文档跑，然后发现文档过时了」这项能力的练习。

今天到这里就够了

不要在 M4 上启动完整训练。第 7 章会讲清楚为什么，以及怎么自己算出「本地要跑多久」这个数。PO 到此结束。三个仓库跑通、踩坑日志有内容，你就具备了「拿到一个陌生研究仓库、两小时内让它出第一个数字」的能力——那正是第 8 周 L0.5 工程关要验的东西。

06 · P1 : MuJoCo 机械臂 IK

周期 2 周，全程本地免费，不需要 GPU。交付物：一个能跑脚本，加一篇「仿真到底在算什么」的笔记。

正运动学（知道关节角，求手在哪）很简单。逆运动学（知道手要去哪，求关节角）才是真问题——而且它是后面所有操作任务的地基：策略输出的动作，最终都要落到关节上。

6.1 加载一条真实的机械臂

① 敲什么

```
$ cd ~/code/mujoco-play && source .venv/bin/activate
$ cat > arm.py <<'EOF'
import mujoco, numpy as np

PATH = "../mujoco_menagerie/universal_robots_ur5e/scene.xml"
model = mujoco.MjModel.from_xml_path(PATH)
data = mujoco.MjData(model)

print("关节数 nq =", model.nq, " 执行器数 nu =", model.nu)
for i in range(model.nbody):
    print(i, mujoco.mj_id2name(model, mujoco.mjtObj.mjOBJ_BODY, i))
EOF

$ python arm.py
```

未实测

② 应该看到

UR5e 是 6 自由度，所以 `nq = 6` 左右；然后是一串 body 名字，最后几个通常是 `wrist_3_link`、`attachment_site` 之类。记下末端那个 body 的名字和序号，下一步要用。

③ 没看到怎么办

文件找不到 → `ls ../mujoco_menagerie/` 看实际目录名，各机器人的文件夹名不完全统一，有的入口叫 `scene.xml` 有的叫 `ur5e.xml`。

想换一条臂 → `franka_emika_panda`（7 自由度）也很好，甚至可以用 `trs_so_arm100`（和你 P3 要买的 SO-101 同系列，提前熟悉）。

6.2 正运动学：给关节角，问手在哪

① 敲什么

```
$ cat > fk.py <<'EOF'
import mujoco, numpy as np

model = mujoco.MjModel.from_xml_path("../mujoco_menagerie/universal_robots_ur5e/scene.xml")
data = mujoco.MjData(model)

EE = model.nbody - 1          # 先用最后一个 body 当末端，6.1 里看到名字后可以改成具体 id

for q in [np.zeros(model.nq), np.full(model.nq, 0.3), np.full(model.nq, -0.5)]:
    data.qpos[:] = q
    mujoco.mj_forward(model, data)    # 只算运动学，不推进时间
    print(np.round(q, 2), "-> 末端位置", np.round(data.xpos[EE], 4))
EOF

$ python fk.py
```

② 应该看到

三行，每行左边是关节角、右边是末端的 xyz 坐标。关节角一变，末端位置就跟着变——这就是正运动学。

这里有个值得停下来想的点

`mj_forward` 和第 4 章的 `mj_step` 不一样：`mj_forward` 只根据当前状态算一遍位置/速度等派生量，不推进时间也不算力；`mj_step` 才是真的往前走一个 timestep。

搞混这两个，是新手「为什么我的机器人不动」和「为什么它自己掉下去了」的常见来源。

6.3 逆运动学：给目标点，求关节角

思路（这就是雅可比迭代法，本质上是牛顿法）：

1. 算当前末端位置和目标的误差
2. 求雅可比矩阵——它回答「每个关节动一点点，末端会往哪个方向移动多少」
3. 解一个线性方程，得到「关节该怎么动」
4. 走一小步，回到第 1 步，直到误差足够小

① 敲什么

```
$ cat > ik.py <<'EOF'
import mujoco, numpy as np

model = mujoco.MjModel.from_xml_path("../mujoco_menagerie/universal_robots_ur5e/scene.xml")
data = mujoco.MjData(model)
EE = model.nbody - 1

target = np.array([0.4, 0.2, 0.4]) # 想让末端去的地方
jacp = np.zeros((3, model.nv)) # 位置雅可比: 3 行(xyz) × nv 列(每个自由度)
jacr = np.zeros((3, model.nv)) # 姿态雅可比, 这里用不上但接口要求传
damp = 1e-3 # 阻尼, 防止奇异位形时解爆炸

for step in range(200):
    mujoco.mj_forward(model, data)
    err = target - data.xpos[EE]
    if np.linalg.norm(err) < 1e-4:
        print(f"第 {step} 步收敛")
        break
    mujoco.mj_jacBody(model, data, jacp, jacr, EE)
    # 阻尼最小二乘: dq = J^T (J J^T + λI)^-1 err
    JJt = jacp @ jacp.T + damp * np.eye(3)
    dq = jacp.T @ np.linalg.solve(JJt, err)
    data.qpos[:model.nv] += 0.5 * dq # 0.5 是步长, 太大容易震荡

mujoco.mj_forward(model, data)
print("目标 ", np.round(target, 4))
print("实际到达", np.round(data.xpos[EE], 4))
print("残差 ", round(float(np.linalg.norm(target - data.xpos[EE])), 6))
print("关节角 ", np.round(data.qpos[:model.nv], 3))
EOF

$ python ik.py
```

未实测

② 应该看到

提示在某一步收敛, 然后目标和实际到达的坐标几乎相同, 残差是个很小的数 (`1e-5` 量级)。

③ 没看到怎么办

不收敛 / 残差一直很大 → 目标点可能在机械臂够不到的地方。把 `target` 改成 6.2 里正运动学算出来的某个真实位置 (那一定可达), 先验证算法本身对不对, 再挑战难的点。

数值爆炸成 nan → 步长 `0.5` 太大, 或者 `damp` 太小。把步长降到 `0.1` 试试。

形状不匹配报错 → 注意 `model.nq` (位置维度) 和 `model.nv` (速度/自由度维度) 对于带自由关节的模型是不相等的。机械臂通常相等, 但这个区别以后会咬你。

这一步真正的功课

把上面的 `damp` 和步长各改三个值，记录收敛步数。然后回答：阻尼调大，收敛变快还是变慢？为什么它能防止「奇异位形」下解爆炸？

能把这个讲清楚，你就理解了后面所有基于优化的控制方法的共同套路。这也正是「笔记」该写的内容——不是抄定义，是写你改参数看到的现象。

6.4 轨迹：让它连续动起来

把 IK 解出来的关节角当作目标，在起点和终点之间插值，逐步推进仿真，用第 4.3 的 viewer 看。交付物就是这个脚本加那篇笔记。

笔记要回答 `PLAN.md` 里 P1 的那个问题——「仿真到底在算什么」：timestep 决定什么、接触力从哪来、求解器迭代次数影响什么、为什么仿真里成功的策略到真机上会失败。最后一问是 sim2real gap 的入口，也是你未来主战场的核心问题之一。

07 · P2：租 GPU 跑真训练

周期 3 周。这一章分两半：本地做什么（不花钱），云上做什么（按小时计费）。

7.1 先自己算出「本地要跑多久」

我不告诉你一个数字，因为我没有在你机器上测过。你自己测，五分钟就有答案——而且这个估算动作本身就是研究者的日常功夫。

① 敲什么

```
$ cd ~/code/lerobot-play && source .venv/bin/activate
# 参数名以 lerobot-train --help 为准, 下面是示意
$ time lerobot-train \
  --policy.type=act \
  --dataset.repo_id=lerobot/aloha_sim_transfer_cube_human \
  --steps=100 \
  --output_dir=/tmp/act-probe
```

未实测

② 应该看到

先下数据集（几百 MB 到几 GB），然后跑 100 步，最后 `time` 打印出总耗时。
然后做这道算术：ACT 原论文级别的训练通常在 10 万步量级。

本地总耗时 $\approx (100 \text{ 步用了几秒}) \div 100 \times 100000 \div 3600 \text{ 小时}$

如果 100 步用了 60 秒，那就是 $60/100 \times 100000 / 3600 \approx 16.7$ 小时——而且那是在你的 Mac 全程不干别的、不睡眠的前提下。这个数字会让「要不要租卡」这个决定变得非常容易。

③ 没看到怎么办

内存不够被系统杀掉（终端显示 `Killed` 或进程无声消失）→ 把 batch size 调到最小再测。16GB 统一内存要同时供 CPU 和 GPU，很容易触顶。

报 MPS 相关的算子不支持 → 常见，LeRobot 的部分算子在 MPS 上没实现。加参数强制用 CPU 再测（更慢，但能测出数量级）。这本身就是「为什么要租卡」的一部分答案。

7.2 本地该做的三件事（不需要 GPU）

对应 `experiments/README.md` 里 P2 的「读 → 改 → 修」三级台阶。这三件事在 Mac 上做，比在租来的卡上做便宜得多，而且学到的更多。

台阶	做什么	为什么
读	找到 ACT 的 <code>forward</code> ，手画一张图：数据从 dataloader 出来后，每一步形状怎么变，一直到 loss。	画完这一张，你才算真的看懂代码，而不是「看着眼熟」。这张图是 L1 复现关的验收物之一。
改	只改一个东西（chunk size / 相机数 / episode 数），跑 100 步，看 loss 曲线怎么变。一次只改一个。	一次改两个，你就永远不知道是哪个起的作用。这是实验设计的第一课。
修	故意把某个张量的形状改错，跑，读 traceback，自己修回来。做 3 次，每次换一种错法。	建立「报错 → 定位」的反射。这个反射是 P3-P5 能不能推进的唯一门槛。

7.3 租哪家

平台	适合谁
AutoDL	国内。支付、网络延迟、下载 HuggingFace 数据集都最省事，界面中文。在深圳的话从这家开始。
RunPod	海外，卡型多，按秒计费。需要外币卡。
vast.ai	个人闲置算力市场，最便宜，但机器质量参差、可能被抢占。适合不怕折腾时。

卡怎么选：ACT / Diffusion Policy 这个量级，单张 24GB 显存的卡（3090 / 4090 一类）足够。不要一上来租 A100 —— 贵，而且你这个阶段的瓶颈不在算力，在你还不熟悉流程。

7.4 三条命令就够了

① 敲什么

```
# 1. 上机（端口和地址平台会给你）
$ ssh -p <端口> root@<地址>

# 2. 开一个后台会话 —— 长任务必须在里面跑
$ tmux new -s train
#      ctrl-b 然后按 d 退出（任务继续跑）；tmux a -t train 回来

# 3. 把本地代码同步上去（在你的 Mac 上跑这条，不是在服务器上）
$ rsync -avz -e "ssh -p <端口>" ./ root@<地址>:/root/work/
```

③ 最容易犯的错

不开 tmux 直接跑训练。网一断、笔记本合盖、Wi-Fi 切换 —— ssh 断开，训练进程跟着死，8 小时白跑。这是租卡阶段最贵的一个错，几乎每个人都会犯一次。养成习惯：ssh 上去第一件事就是 `tmux new`。

三条纪律

数据放机器上（每次重下太慢）· 代码用 git 同步（rsync 只用于临时改动）· checkpoint 定期 rsync 回本地（实例一释放，盘上的东西就没了——包括你跑了一整夜的结果）。

7.5 跑起来，然后把结果带回来

① 敲什么（在服务器的 TMUX 里）

```
$ lerobot-train --policy.type=act \
  --dataset.repo_id=<数据集> \
  --output_dir=/root/work/outputs/act-run1

# 另开一个窗口看显存占用
$ nvidia-smi
```

② 应该看到

loss 稳定下降；`nvidia-smi` 显示显存被占用、GPU 利用率不是 0。利用率长期是 0 说明瓶颈在数据加载不在算力，那就该调 dataloader 的进程数，而不是换更贵的卡。

① 敲什么（训练结束后，在你的 MAC 上）

```
$ rsync -avz -e "ssh -p <端口>" \
  root@<地址>:/root/work/outputs/ ./outputs/
```

P2 的验收

公开数据集上复现的指标落在原文 $\pm 10\%$ 内；一份复现报告；ACT 与 Diffusion Policy 的失效模式对照表；还有 7.2 里那张形状变化图。

指标对不上不是失败——说不清为什么对不上才是。差在哪、你怀疑是什么、怎么验证，这三句写清楚，报告就有价值。

08 · 卡住了怎么办

8.1 60 秒规则的操作版

报错先自己读 60 秒再问 AI。这 60 秒具体怎么用：

秒	做什么	回答什么问题
0–15	从最后一行往上读。最后一行是错误类型和信息。	什么坏了
15–40	往上找第一个属于你的文件路径——不在 <code>site-packages/</code> 里、也不在 <code>.venv/</code> 里的那一行。	哪儿坏了
40–60	对照下面的表，看是不是四类高频错之一。	要不要问人

中间那一大堆库内部的栈帧，全部跳过。新手最容易犯的错是从第一行开始一行行读——那些几乎都是别人的代码，和你无关。

8.2 高频报错对照表

报错	真实原因与处理
<code>ModuleNotFoundError</code> 且模块名 = 仓库名	忘了 <code>uv pip install -e .</code> ，或者环境没激活。先看提示符前面有没有 <code>(...)</code> 。不要先怀疑 Python 版本。
<code>command not found</code> (<code>brew</code> / <code>lerobot-info</code> / <code>uv</code>)	PATH 问题或环境没激活。 <code>brew</code> 找不到 → 第 2.2 节那两行 PATH 配置没做。命令行工具找不到 → 重开终端，或 <code>source .venv/bin/activate</code> 。
<code>RuntimeError: ... shapes cannot be multiplied</code>	张量形状对不上。在报错那一行前面插 <code>print(x.shape)</code> ，往上追到第一个形状不对的地方。这是第 7.2 节「修」那一级要反复练的。
进程无声消失 / 显示 <code>Killed</code>	内存被系统杀了。Mac 上是统一内存吃满。调小 batch size，或者关掉浏览器再试。
<code>CUDA out of memory</code> (租的卡上)	batch 太大 / 推理时忘了 <code>torch.no_grad()</code> / 显存被别的进程占着。先 <code>nvidia-smi</code> 看一眼。
必须用 <code>mjpython</code>	macOS 的 MuJoCo 交互式窗口限制，见 4.3。不是你装错了。
找不到 <code>libpython3.12.dylib</code>	<code>mjpython</code> 和 uv 自带 Python 的已知冲突。用 Homebrew 的 Python 重建环境，见 4.3。
下载卡住不动	网络。 <code>Control + C</code> 中断重跑，uv 和 huggingface 都会复用已下载的部分，不会从头开始。

8.3 什么时候该停

时间盒到了就停，这是规则不是建议

每章都标了时间盒。到点没通，把当前状态写进踩坑日志，收工。

不要做的两件事：硬扛到半夜（第二天状态差，损失远大于今天多跑通一个仓库）；为了跑通去抄一个别人的现成环境（那等于把这次训练作废——PO 练的就是自己把环境搞对的能力）。

记住 `experiments/P0-toolchain/README.md` 里那句：卡住的过程本身就是交付物。跑通了但日志是空的，那一天不算完成。

8.4 问 AI 的正确姿势

可以	不可以
贴报错让它解释「这句话什么意思」	让它直接给你一份能跑的环境配置，你不看就用
让它解释某段代码在干什么	让它替你读论文并给结论（前 6 个月）
让它出题考你、扮演审稿人反驳你	用它跳过 60 秒规则
让它写画图和数据处理的脚手架代码	让它改完你不看就 commit（违反逐行可复述）

分界线：AI 可以替你打字，不能替你判断。

09 · 命令速查卡

这一页可以单独打印贴在显示器旁边。

终端

```
$ pwd                # 我在哪
$ ls -la             # 这里有什么（含隐藏文件）
$ cd ~/code/ai-study-lab # 去某处（Tab 键补全！）
Control + C         强行中止
```

环境（每个项目一间房）

```
$ uv venv --python 3.12          # 建房间
$ uv venv --python /opt/homebrew/bin/python3.12 # MuJoCo 用这个
$ source .venv/bin/activate      # 进房间（看提示符前的括号！）
$ uv pip install -e .           # 装当前仓库
$ uv pip install "lerobot[dataset]" # 装带可选组的包
$ deactivate                    # 出房间
$ rm -rf .venv                  # 搞砸了就删掉重建，不要手工修
```

三个仓库的第一个数字

```
$ python smoke.py          # micrograd : 三个梯度数字
$ python drop.py           # MuJoCo : 盒子落下, z 变小
$ lerobot-info             # LeRobot : 环境报告, MPS 可用
$ mjpgpython watch.py      # MuJoCo 交互窗口（注意不是 python）
```

租卡

```
$ ssh -p <端口> root@<地址>
$ tmux new -s train        # 上去第一件事! ctrl-b d 退出, tmux a -t train 回来
$ nvidia-smi              # 看显存和利用率
$ rsync -avz -e "ssh -p <端口>" ./ root@<地址>:/root/work/ # 推代码
$ rsync -avz -e "ssh -p <端口>" root@<地址>:/root/work/outputs/ ./outputs/ # 拉结果
```

卡住时的顺序

1. 最后一行说什么？ # 什么坏了
2. 第一个属于我的文件在哪？ # 哪儿坏了（跳过 site-packages）
3. 提示符前面有括号吗？ # 环境激活了吗 — 一半的问题在这
4. pwd 对吗？ # 我在正确的目录吗
5. 满 60 秒了再问 AI

每天收工前

```
$ cd ~/code/ai-study-lab
# 写今天的笔记（我原以为... 现在知道... 还不懂...）
$ open notes/          # 或用任何编辑器
# 踩坑记进 experiments/P0-toolchain/踩坑日志.md
$ git add -A && git commit -m "notes: ..." && git push
```

如果只记住三句话

1. 提示符前面没有括号，就是环境没激活 —— 这一条能解释你一半的报错。
2. 时间盒到了就停，把卡住的过程记下来 —— 那是交付物，不是失败。
3. 报错先自己读 60 秒 —— 这 60 秒练的是别人替你练不了的肌肉。